

Parul University

Name: jay patil

Email: 2503031460586@paruluniversity.ac.in

Roll no: 2503031460586

Phone: 7016550648

Branch: Parul University

Department: AIML3_Batch 2

Batch: 2029

Degree: B.Tech - AI & ML

Scan to verify results



PIET_OOPs Using C++_Course

PIET_Pointers_Session 11_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Deva, a teacher, needs a program to track student attendance using a class named Student. The class should have data members for roll number, total classes, and classes attended.

The program will use pointers to Student objects to input data for 5 students, calculate their attendance percentage, and display it formatted to two decimal places. Handle division by zero by setting the percentage to 0.00% when the total number of classes is zero.

Formula used: Attendance Percentage = (Classes Attended / Total Classes) × 100

Input Format

The input consists of five lines, each containing three space-separated integers, representing the roll number, total number of classes, and number of classes attended.

Output Format

For each student, output a line in the following format: "Roll No: X Attendance: Y %". Where X is the student's roll number and Y is the attendance percentage rounded to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 123 3 2

111 0 0

101 10 8

132 8 7

167 7 7

Output: Roll No: 123 Attendance: 66.67%

Roll No: 111 Attendance: 0.00%

Roll No: 101 Attendance: 80.00%

Roll No: 132 Attendance: 87.50%

Roll No: 167 Attendance: 100.00%

Answer

```
// You are using GCC
```

```
#include <iostream>
```

```
#include <iomanip>
```

```
class Student {
```

```
public:
```

```
    int rollNumber;
```

```
    int totalClasses;
```

```
    int classesAttended;
```

```
    void displayAttendance() {
```

```
        double percentage = 0.00;
```

```
        if (totalClasses > 0) {
```

```
            percentage = (static_cast<double>(classesAttended) / totalClasses) *
```

```
            100.0;
```

```

    }
    std::cout << "Roll No: " << rollNumber
    << " Attendance: " << std::fixed << std::setprecision(2)
    << percentage << "%" << std::endl;
}
};

int main() {
    Student* students[5];

    for (int i = 0; i < 5; ++i) {
        students[i] = new Student();
        if (!(std::cin >> students[i]->rollNumber >> students[i]->totalClasses >>
students[i]->classesAttended)) {
            break;
        }
    }

    for (int i = 0; i < 5; ++i) {
        students[i]->displayAttendance();
        // Clean up memory
        delete students[i];
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Jeeva, a theater manager, needs to organize and track seating arrangements using a SeatingArrangement class. Each row of seats has a capacity, occupied seats, and a row number.

Using pointers to SeatingArrangement objects, Jeeva will input the details for each row, calculate the available seats, and check if the row is completely empty. The program should display each row's number, and the number of available seats, and indicate if the row is empty.

Input Format

The first line consists of an integer n, representing the number of rows.

The next n line contains three space-separated integers, representing the row number, seating capacity, and number of occupied seats.

Output Format

For each row, the output displays a line in the following format:

"Row No: [Row Number] | Available Seats: [Available Seats] | Empty Row: [Yes/No]".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 10 3

2 15 6

3 12 0

Output: Row No: 1 | Available Seats: 7 | Empty Row: No

Row No: 2 | Available Seats: 9 | Empty Row: No

Row No: 3 | Available Seats: 12 | Empty Row: Yes

Answer

```
// You are using GCC
```

```
#include <iostream>
```

```
#include <string>
```

```
class SeatingArrangement {
```

```
private:
```

```
    int rowNumber;
```

```
    int capacity;
```

```
    int occupied;
```

```
public:
```

```
    SeatingArrangement(int r, int c, int o) : rowNumber(r), capacity(c), occupied(o)
```

```
    {
```

```

int getAvailableSeats() {
    return capacity - occupied;
}

std::string isEmpty() {
    return (occupied == 0) ? "Yes" : "No";
}

void displayInfo() {
    std::cout << "Row No: " << rowNumber
        << " | Available Seats: " << getAvailableSeats()
        << " | Empty Row: " << isEmpty() << std::endl;
}
};

int main() {
    int n;
    if (!(std::cin >> n)) return 0;

    SeatingArrangement* rows[10];

    for (int i = 0; i < n; ++i) {
        int r, c, o;
        std::cin >> r >> c >> o;
        rows[i] = new SeatingArrangement(r, c, o);
    }

    for (int i = 0; i < n; ++i) {
        rows[i]->displayInfo();
        delete rows[i];
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Amit is a supervisor in a company where employees receive additional pay for overtime work. There are two types of employees: Full-Time and Part-

Time. A Full-TimeEmployee receives a fixed salary, and overtime is paid at Rs 200 per hour.

A Part-TimeEmployee is paid based on total hours worked at Rs 300 per hour. Implement a program using pointers to a derived class where the base class Employee has a virtual function calculatePay(). The derived classes FullTimeEmployee and PartTimeEmployee should override this function to compute the total salary based on the given inputs.

Input Format

The first line of input consists of an integer 'type', representing the employee type (1 for Full-Time, 2 for Part-Time).

The second line of input consists of an integer 'fixedSalary' representing the fixed salary (only for Full-Time employees).

The third line of input consists of an integer 'overtimeHours' representing the number of overtime hours (only for Full-Time employees).

The fourth line of input consists of an integer 'totalHours' representing the total working hours (only for Part-Time employees).

Output Format

The output displays the total salary in the following format:

"Rs. <total_salary>" (rounded to 2 decimal places).

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

20000

10

Output: Rs. 22000.00

Answer

```
// You are using GCC
```

```
#include <iostream>
```

```
#include <iomanip>
```

```
class Employee {
```

```
public:
```

```
    virtual double calculatePay() = 0;
```

```
    virtual ~Employee() {}
```

```
};
```

```
class FullTimeEmployee : public Employee {
```

```
private:
```

```
    int fixedSalary;
```

```
    int overtimeHours;
```

```
public:
```

```
    FullTimeEmployee(int salary, int hours) : fixedSalary(salary),  
    overtimeHours(hours) {}
```

```
    double calculatePay() override {
```

```
        return fixedSalary + (overtimeHours * 200.0);
```

```
    }
```

```
};
```

```
class PartTimeEmployee : public Employee {
```

```
private:
```

```
    int totalHours;
```

```
public:
```

```
    PartTimeEmployee(int hours) : totalHours(hours) {}
```

```
    double calculatePay() override {
```

```
        return totalHours * 300.0;
```

```
    }
```

```
};
```

```
int main() {
```

```
    int type;
```

```
    std::cin >> type;
```

```
    Employee* emp = nullptr;
```

```
    if (type == 1) {
```

```
        int salary, hours;
```

```
        std::cin >> salary >> hours;
```

```
        emp = new FullTimeEmployee(salary, hours);
    } else if (type == 2) {
        int totalHours;
        std::cin >> totalHours;
        emp = new PartTimeEmployee(totalHours);
    }

    if (emp != nullptr) {
        std::cout << "Rs. " << std::fixed << std::setprecision(2) << emp-
>calculatePay() << std::endl;
        delete emp; // Free memory
    }

    return 0;
}
```

Status : Correct

Marks : 10/10